

Qualidade de repositórios Java populares

Pedro Negri Leão Lambert

09/04/2026

Link para repo/dados:

<https://github.com/Pedro-nll/LabExperimentacaoDeSoftware/tree/enunciado2>

Qualidade de repositórios Java populares.....	1
Introdução.....	2
Contextualização.....	2
Restrições e riscos.....	2
Problema foco do experimento.....	3
Questões de Pesquisa.....	3
Hipóteses.....	4
Objetivo.....	5
Objetivo principal.....	5
Objetivos específicos.....	5
Metodologia.....	5
Passo a passo do experimento.....	5
Etapa A – Coleta dos repositórios.....	6
Etapa B – Preparação da ferramenta de análise.....	6
Etapa D – Consolidação das métricas.....	7
Etapa E – Análise exploratória e geração de gráficos.....	7
Etapa F – Monitoramento da execução.....	8
Decisões.....	9
Materiais utilizados.....	10
Métodos utilizados.....	10
Métricas e suas unidades.....	10
Visualização dos Resultados.....	11
Discussão dos Resultados.....	14
Conclusão.....	15
Trabalhos futuros.....	16
Reprodutibilidade.....	16

Introdução

Contextualização

O desenvolvimento de software open-source tem se consolidado como um dos principais modelos de produção de software na indústria contemporânea. Projetos amplamente utilizados, como bibliotecas, frameworks e sistemas completos, são frequentemente mantidos por comunidades distribuídas de desenvolvedores, com contribuições contínuas ao longo do tempo.

Nesse contexto, surge um desafio relevante: garantir a qualidade interna do código ao longo da evolução do sistema. Em ambientes colaborativos, diferentes desenvolvedores contribuem com estilos, níveis de experiência e práticas distintas, o que pode impactar atributos como coesão, acoplamento e complexidade estrutural do software.

Para lidar com esse desafio, ferramentas de análise estática, como a CK, permitem mensurar atributos internos do código a partir de métricas clássicas de engenharia de software, como CBO (acoplamento), DIT (profundidade de herança) e LCOM (coesão). Essas métricas fornecem uma base quantitativa para avaliar a qualidade estrutural de sistemas.

Este experimento busca analisar repositórios Java populares no GitHub, correlacionando métricas de qualidade interna com características do processo de desenvolvimento, como popularidade, maturidade, atividade e tamanho. A análise visa identificar padrões e relações que possam indicar fatores associados à manutenção da qualidade em projetos open-source.

Restrições e riscos

Algumas limitações devem ser consideradas na condução deste experimento.

Primeiramente, o estudo foi restrito a repositórios escritos em Java, o que limita a generalização dos resultados para outras linguagens e ecossistemas. Diferentes linguagens podem apresentar padrões distintos de arquitetura e desenvolvimento.

Outro ponto relevante diz respeito às métricas de qualidade utilizadas. As métricas CK são calculadas a partir de análise estática de código e representam proxies de atributos de qualidade. No entanto, elas não capturam aspectos como legibilidade, qualidade de design arquitetural em alto nível ou práticas de engenharia adotadas pela equipe.

Por fim, a análise é baseada em agregações ao nível de repositório (média, mediana e desvio padrão das métricas por classe), o que pode mascarar variações internas significativas entre diferentes partes do código.

Problema foco do experimento

Este experimento busca investigar como características do processo de desenvolvimento de repositórios open-source estão relacionadas com atributos internos de qualidade do código.

Mais especificamente, o problema central consiste em entender se fatores como popularidade, maturidade, atividade e tamanho dos repositórios possuem alguma relação com métricas estruturais de qualidade, como acoplamento, herança e coesão.

Essa investigação é relevante pois permite avaliar se práticas associadas ao crescimento e à evolução de projetos open-source impactam positivamente ou negativamente a qualidade interna do software.

Questões de Pesquisa

Com base no problema definido, o estudo busca responder às seguintes questões de pesquisa:

RQ	Descrição	Métrica do repositório
RQ1	Qual a relação entre qualidade e popularidade?	Número de estrelas
RQ2	Qual a relação entre qualidade e maturidade?	Idade do repositório
RQ3	Qual a relação entre qualidade e atividade?	Days since push
RQ4	Qual a relação entre qualidade e tamanho?	LOC

Hipóteses

Com base na literatura e na intuição sobre desenvolvimento de software, foram formuladas as seguintes hipóteses:

Hipótese	Descrição
H1	Popularidade não está diretamente associada à qualidade interna
H2	Repositórios mais antigos tendem a apresentar pior qualidade estrutural
H3	Maior atividade dificulta a manutenção da qualidade
H4	Repositórios maiores tendem a apresentar menor coesão e maior complexidade

H1 – Popularidade não está diretamente associada à qualidade interna

A popularidade de um repositório está mais relacionada à sua utilidade percebida, adoção pela comunidade ou valor educacional do que necessariamente à sua qualidade estrutural. Projetos amplamente utilizados podem acumular complexidade interna ao longo do tempo sem que isso impacte diretamente sua adoção.

H2 – Repositórios mais antigos tendem a apresentar pior qualidade estrutural

Sistemas mais antigos estão sujeitos ao acúmulo de dívida técnica decorrente de mudanças contínuas, evolução de requisitos e possíveis limitações das decisões de design iniciais. Esse processo pode levar ao aumento de acoplamento e redução de coesão ao longo do tempo.

H3 – Maior atividade dificulta manutenção da qualidade

Repositórios com alta atividade apresentam maior volume de alterações, contribuições externas e evolução rápida do código. Esse dinamismo pode dificultar a manutenção consistente de padrões de qualidade, especialmente em ambientes com múltiplos contribuidores.

H4 – Repositórios maiores tendem a apresentar menor coesão e maior complexidade

À medida que o tamanho do sistema cresce, aumenta também a complexidade estrutural e a dificuldade de manter modularidade adequada. Sistemas maiores tendem a apresentar maior acoplamento entre componentes e menor coesão interna, tornando a manutenção mais desafiadora.

Objetivo

Objetivo principal

Analisar a relação entre características do processo de desenvolvimento e atributos internos de qualidade em repositórios Java populares no GitHub.

Objetivos específicos

- Correlacionar métricas de qualidade com popularidade, maturidade, atividade e tamanho
- Identificar padrões e possíveis relações entre essas dimensões
- Avaliar as hipóteses propostas com base nos resultados obtidos

Metodologia

Passo a passo do experimento

O experimento foi conduzido por meio de um pipeline automatizado de coleta, processamento e análise de dados de repositórios Java populares no GitHub. O pipeline foi estruturado em seis etapas principais, garantindo reprodutibilidade e robustez na execução.

Etapa	Descrição	Entrada	Saída
A	Coleta de repositórios	API GitHub	<code>java_top1000.csv</code>
B	Preparação da ferramenta CK	Repositório CK	JAR executável
C	Execução do CK	Repositórios + CK	Dados por repositório
D	Consolidação das métricas	Arquivos <code>class.csv</code>	<code>ck_summary.csv</code>
E	Geração de gráficos	Dataset consolidado	Gráficos + correlações
F	Monitoramento	Logs + outputs	Relatório de progresso

Etapa A – Coleta dos repositórios

A coleta dos dados foi realizada utilizando a API de busca do GitHub, com o objetivo de obter os 1000 repositórios Java mais populares, definidos com base no número de estrelas.

Para garantir robustez e completude da coleta, foram adotadas as seguintes estratégias:

- Paginação fixa de 10 repositórios por requisição
- Mecanismo de retry com backoff para falhas temporárias
- Uso de checkpoints para permitir retomada da coleta
- Remoção de duplicatas com base no identificador `owner/repo`

O resultado dessa etapa é um arquivo CSV contendo os metadados dos repositórios analisados.

Etapa B – Preparação da ferramenta de análise

A ferramenta CK foi utilizada para extrair métricas de qualidade estrutural do código. O processo incluiu:

- Clonagem ou atualização do repositório da ferramenta
- Build do projeto utilizando Maven
- Geração de um JAR standalone para execução automatizada

Essa etapa garante consistência na análise de todos os repositórios.

Etapa C – Execução da análise estática

Cada repositório foi processado individualmente pela ferramenta CK. Para tornar essa etapa viável em larga escala, foram adotadas diversas estratégias operacionais:

- Clonagem superficial (*shallow clone*, `depth=1`) para reduzir uso de rede e armazenamento
- Execução paralela com múltiplos workers
- Registro incremental de logs em arquivo (`pipeline_run_log.csv`)

O sistema de logging foi projetado para permitir retomada segura da execução:

- Cada execução (sucesso ou falha) é registrada imediatamente
- O estado mais recente de cada repositório é considerado a fonte de verdade
- Repositórios já processados com sucesso são ignorados em execuções futuras
- Repositórios com falha permanecem elegíveis para reprocessamento

Após o processamento, os repositórios clonados são removidos para reduzir o uso de armazenamento.

Etapa D – Consolidação das métricas

Os dados gerados pela ferramenta CK, originalmente no nível de classe, foram agregados para o nível de repositório.

Foram calculadas as seguintes estatísticas para cada métrica:

- Média
- Mediana
- Desvio padrão

Além disso, foi realizado tratamento de dados inválidos:

- Valores não finitos (NaN ou infinito) foram descartados
- Métricas sem valores válidos foram definidas como 0.0
- Conjuntos com único valor receberam desvio padrão igual a 0.0

Essa etapa garante consistência estatística e comparabilidade entre repositórios.

Etapa E – Análise exploratória e geração de gráficos

Os dados de qualidade foram integrados aos metadados dos repositórios e utilizados para gerar gráficos de dispersão para cada combinação entre:

- Variável de processo (RQ)
- Métrica de qualidade

Além disso, foram calculadas correlações:

- Pearson (linear)
- Spearman (monotônica)

Essas medidas foram utilizadas como suporte à interpretação dos padrões observados.

Etapa F – Monitoramento da execução

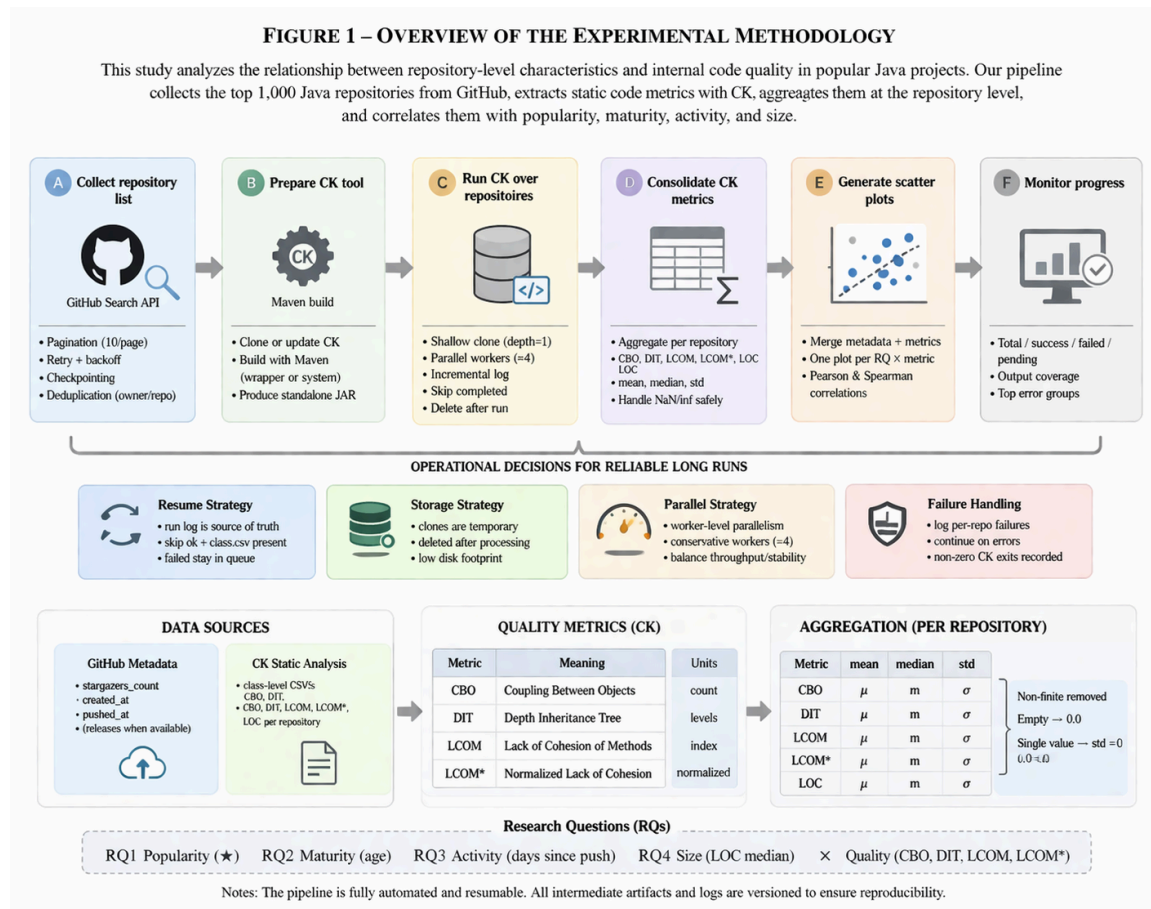
Foi implementado um mecanismo de monitoramento do progresso da execução, baseado em:

- Arquivo de log do pipeline
- Diretório de saída dos resultados

O sistema permite identificar:

- Repositórios processados com sucesso
- Falhas de execução
- Itens pendentes
- Cobertura de resultados gerados

Essa etapa foi essencial para garantir controle e confiabilidade em execuções longas.



Decisões

Durante a execução do experimento, algumas decisões metodológicas foram necessárias para garantir consistência, viabilidade e qualidade dos dados analisados.

A Tabela a seguir resume as principais decisões metodológicas adotadas:

Aspecto	Decisão adotada
Universo	Top 1000 repositórios Java por número de estrelas
Unidade de análise	Repositório (agregação de métricas por classe)
Métrica de atividade	Uso de <i>days since push</i> como proxy (releases indisponíveis)
Métrica de tamanho	Uso de LOC mediano gerado pelo CK
Tratamento de outliers	Uso da mediana como principal medida de tendência central
Valores inválidos	Remoção de valores não finitos (NaN/inf) antes da agregação

Já a tabela abaixo resume as principais decisões operacionais adotadas:

Aspecto	Estratégia adotada	Impacto
Resume	Uso de <code>pipeline_run_log.csv</code> como fonte de verdade	Permite retomada segura
Retry	Reprocessamento de falhas	Aumenta completude dos dados
Storage	Clones temporários e remoção após uso	Reduz uso de disco
Paralelismo	Execução com múltiplos workers (≈ 4)	Balanceia desempenho e estabilidade
Falhas	Log por repositório + continuidade do pipeline	Evita interrupção global
Execução incremental	Skip de repositórios já processados	Evita retrabalho

Materiais utilizados

Os seguintes recursos foram utilizados na condução do experimento:

- Linguagem de programação Python
- API de busca do GitHub
- Ferramenta de análise estática CK
- Ambiente de execução local
- Bibliotecas de análise de dados e visualização

Métodos utilizados

O experimento segue a abordagem de **Mineração de Repositórios de Software (MSR)**, utilizando dados públicos disponíveis no GitHub.

A análise foi conduzida em duas etapas principais:

1. **Extração de métricas**
 - Coleta de metadados dos repositórios (estrelas, datas, atividade)
 - Extração de métricas de qualidade via análise estática (CK)
2. **Análise estatística**
 - Cálculo de média, mediana e desvio padrão por repositório
 - Análise de distribuição dos dados
 - Geração de gráficos de dispersão
 - Cálculo de correlações (Pearson e Spearman)

As correlações foram utilizadas como ferramenta exploratória para identificar possíveis relações entre variáveis, sem assumir causalidade.

Métricas e suas unidades

As métricas utilizadas no experimento foram organizadas em duas categorias: métricas de processo e métricas de qualidade.

Métricas de processo

Métrica	Descrição
Popularidade	Número de estrelas
Maturidade	Idade do repositório
Atividade	Tempo desde último push
Tamanho	Linhas de código

Métricas de Qualidade (CK)

Métrica	Descrição
CBO	Acoplamento entre classes
DIT	Profundidade da árvore de herança
LCOM	Falta de coesão entre métodos
LCOM*	Versão normalizada de coesão

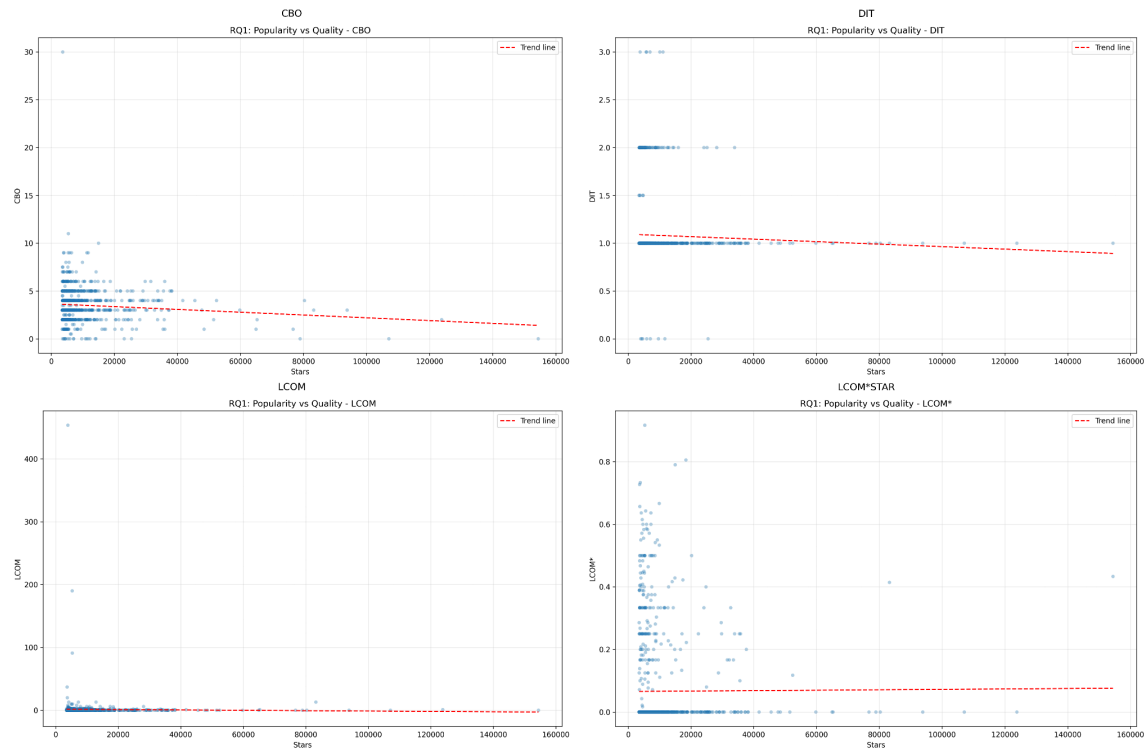
Visualização dos Resultados

Nesta seção são apresentados os resultados obtidos para cada uma das questões de pesquisa, por meio de gráficos de dispersão e análise das correlações entre as variáveis.

Cada gráfico representa a relação entre uma métrica de processo (popularidade, maturidade, atividade ou tamanho) e uma métrica de qualidade (CBO, DIT, LCOM e LCOM*), considerando os valores agregados por repositório.

RQ1 – Popularidade vs Qualidade

rq1_popularity - Combined Scatter Plots

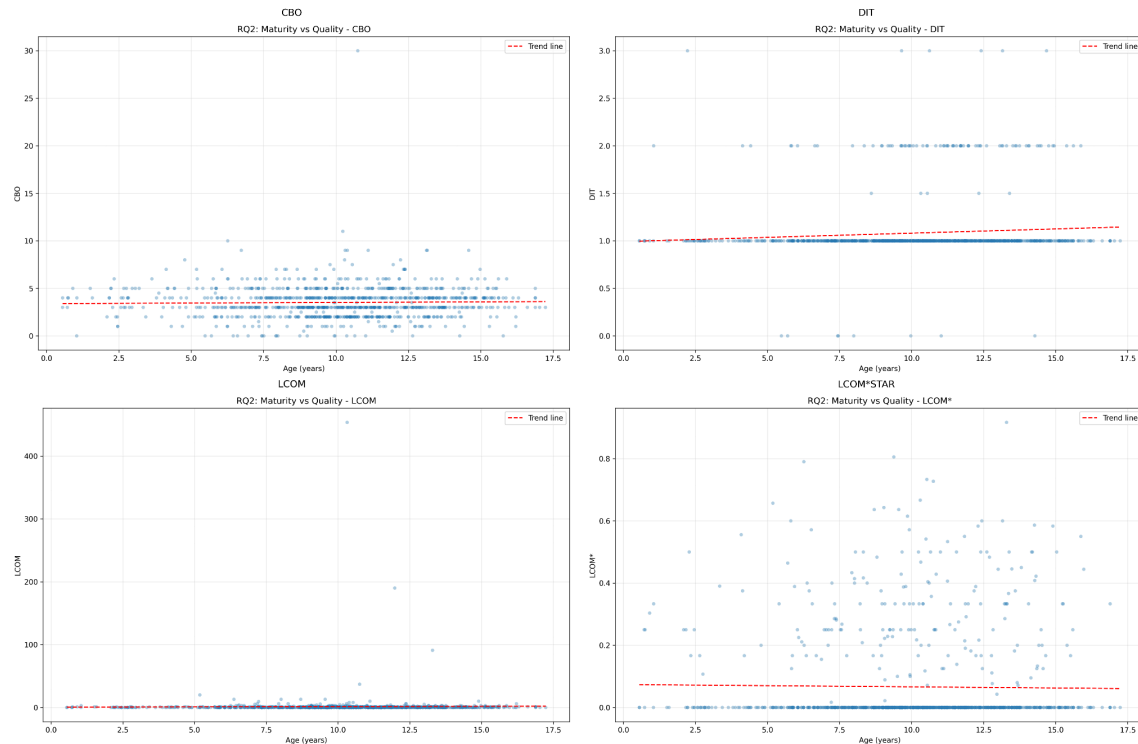


Os gráficos indicam ausência de relação clara entre o número de estrelas e as métricas de qualidade. A distribuição dos pontos apresenta alta dispersão, sem tendência visual consistente.

As correlações calculadas reforçam essa observação, com valores próximos de zero em todas as métricas (por exemplo, CBO: -0.0972, LCOM: -0.0211). Isso indica ausência de relação linear ou monotônica relevante entre popularidade e qualidade interna.

RQ2 – Maturidade vs Qualidade

rq2_maturity - Combined Scatter Plots

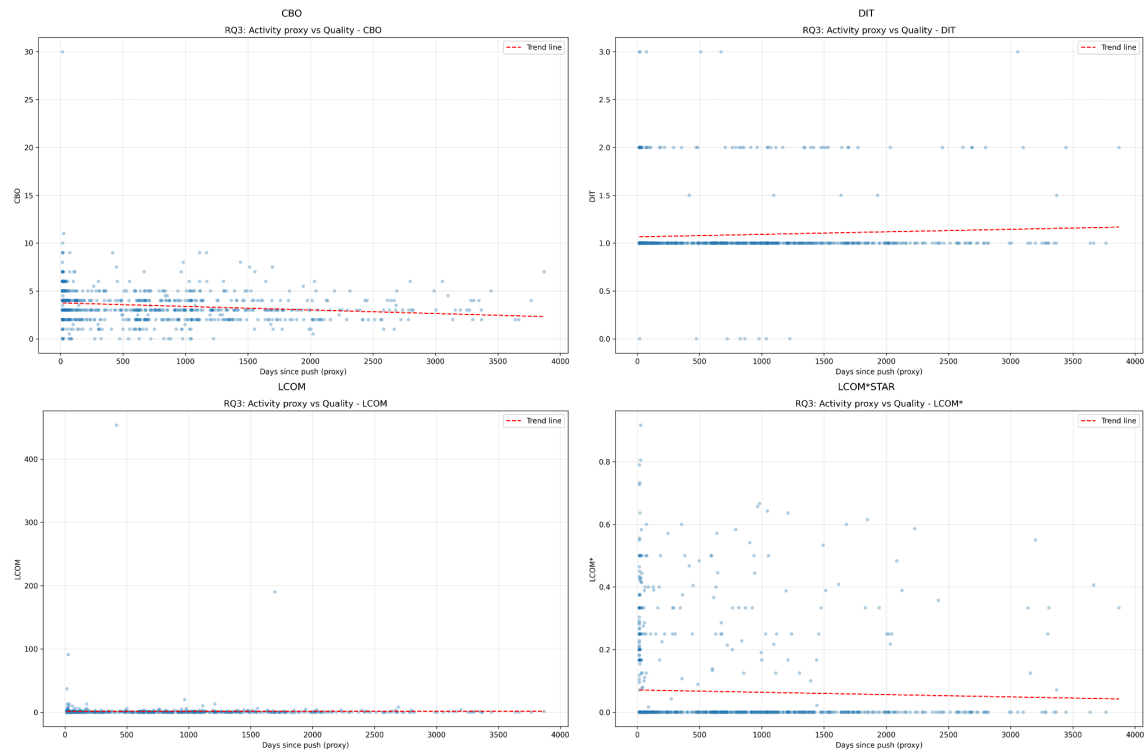


A análise dos gráficos mostra leve tendência de aumento em algumas métricas, porém com alta variabilidade entre os repositórios.

As correlações são fracas (ex.: DIT: 0.0867, LCOM: 0.0163), indicando que a idade do repositório possui impacto limitado na qualidade estrutural. Não há evidência de degradação sistemática da qualidade ao longo do tempo.

RQ3 – Atividade vs Qualidade

rq3_activity - Combined Scatter Plots



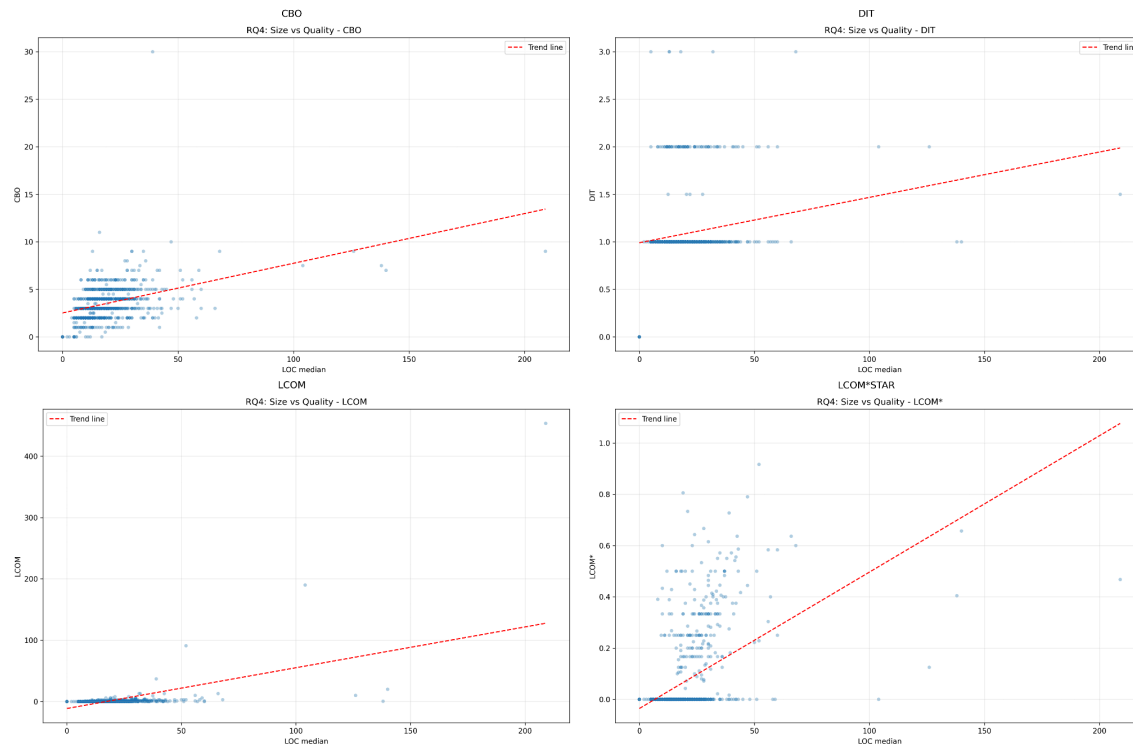
Para esta análise, foi utilizado o tempo desde o último push (*days since push*) como proxy de atividade.

Observa-se uma leve tendência negativa entre atividade e CBO (Spearman ≈ -0.31), sugerindo que repositórios mais ativos tendem a apresentar menor acoplamento. Entretanto, para as demais métricas, não há padrões claros.

De forma geral, a atividade apresenta baixa correlação com qualidade estrutural.

RQ4 – Tamanho vs Qualidade

rq4_size - Combined Scatter Plots



Os gráficos evidenciam uma tendência positiva mais clara entre tamanho e métricas de qualidade, especialmente para LCOM e LCOM*.

As correlações são moderadas ($LCOM \approx 0.55$, $LCOM^* \approx 0.47$), indicando que repositórios maiores tendem a apresentar menor coesão e maior complexidade estrutural.

Esse é o único conjunto de resultados onde um padrão consistente é observado.

Discussão dos Resultados

A análise dos resultados permite avaliar as hipóteses propostas à luz dos dados empíricos obtidos.

A hipótese **H1**, que sugere que popularidade não está diretamente associada à qualidade interna, é fortemente corroborada. As correlações próximas de zero e a ausência de padrões nos gráficos indicam que projetos populares não necessariamente possuem melhor qualidade estrutural.

A hipótese **H2**, que propõe que repositórios mais antigos tendem a apresentar pior qualidade, não foi confirmada. Os resultados mostram correlações fracas entre

maturidade e métricas de qualidade, indicando que a idade do projeto não é um fator determinante para degradação estrutural.

Em relação à hipótese **H3**, esperava-se que maior atividade dificultasse a manutenção da qualidade. No entanto, os resultados mostram um comportamento mais complexo: há indícios de que repositórios mais ativos apresentam menor acoplamento (CBO), o que pode indicar práticas mais frequentes de refatoração. Para as demais métricas, não há evidência clara de impacto.

A hipótese **H4**, por sua vez, foi confirmada. Observa-se uma relação consistente entre o tamanho do repositório e a piora em métricas de coesão e complexidade. Esse resultado é coerente com a literatura, uma vez que sistemas maiores tendem a ser mais difíceis de manter e modularizar adequadamente.

De forma geral, os resultados indicam que características externas do projeto (popularidade, idade e atividade) possuem baixo impacto direto na qualidade estrutural, enquanto fatores internos, como tamanho do sistema, apresentam maior influência.

Conclusão

Este estudo analisou a relação entre características de processo e métricas de qualidade em repositórios Java populares no GitHub.

Os resultados indicam que:

- Popularidade não está associada à qualidade interna
- Maturidade apresenta baixo impacto sobre a qualidade estrutural
- Atividade possui relação limitada, com possível efeito positivo sobre acoplamento
- Tamanho do sistema apresenta relação consistente com aumento de complexidade e redução de coesão

Dentre os fatores analisados, o tamanho do repositório se mostrou o mais relevante para explicar variações nas métricas de qualidade.

Além disso, os resultados reforçam a ideia de que qualidade de software é influenciada principalmente por fatores internos ao sistema, como estrutura e organização do código, e não apenas por características externas do processo de desenvolvimento.

Como limitações, destaca-se a perda de parte dos repositórios devido a falhas na execução da ferramenta CK, resultando em um conjunto final de 967 repositórios analisados. Ainda assim, a amostra permanece suficientemente representativa para análise exploratória.

Trabalhos futuros

Como trabalhos futuros, sugere-se:

- investigar outras métricas de qualidade
- analisar evolução temporal das métricas
- considerar fatores adicionais, como número de contribuidores e práticas de revisão

Reprodutibilidade

O experimento foi estruturado de forma totalmente automatizada, permitindo sua reprodução a partir dos scripts disponibilizados no repositório do projeto.

O pipeline é composto por etapas sequenciais de coleta, processamento e análise de dados, sendo executado por meio dos scripts descritos a seguir.

A reprodução completa pode ser realizada a partir da raiz do repositório:

1. Coleta dos repositórios

```
python enunciado2/scripts/collect_java_repos.py \  
--output enunciado2/data/java_top1000.csv \  
--target 1000 \  
--per-page 10
```

2. Build da ferramenta CK

```
python enunciado2/scripts/setup_ck.py \  
--ck-dir enunciado2/tools/ck \  
--build
```

3. Execução do pipeline de análise

```
python enunciado2/scripts/run_ck_pipeline.py \  
--repos-csv enunciado2/data/java_top1000.csv \  
--sample-size 1000 \  
--workspace enunciado2/work \  
--output enunciado2/output \  
--ck-dir enunciado2/tools/ck \  
--workers 4 \  
--timeout-minutes 30
```

4. Consolidação das métricas

```
python enunciado2/scripts/summarize_ck_metrics.py \  
--input-dir enunciado2/output \  
--output-csv enunciado2/output/ck_summary.csv
```


5. Geração dos gráficos

```
python enunciado2/scripts/plot_rq_scatter.py \  
  --repos-csv enunciado2/data/java_top1000.csv \  
  --ck-summary-csv enunciado2/output/ck_summary.csv \  
  --output-dir enunciado2/output/rq_graphs
```